LAPORAN PRAKTIKUM 05
DESAIN ENDPOINT DAN IMPLEMENTASI WEB SERVICE DENGAN FRAMEWORK

Mata kuliah	Web Service Engineering
Topik	Desain Endpoint dan Implementasi Web Service dengan Framework
Studi kasus	Layanan Katalog Buku Perpustakaan Kampus
Nama	Syariah Nor Alfisah
NIM	240104040130
Kelas	TI 24 A
Dosen pengampu	Muhayat, M.IT
Tanggal praktikum	10 November 2026
Sistem operasi	Windows (Command Prompt)
Browser	Google Chrome
Editor	Visual Studio Code
Node.js / npm	v24.22.2 / 11.19.1 (sebelumnya v24.21.0 / 11.19.0)
Framework	express 5.3.0 dan swagger-ui-express 5.0.1
Alamat layanan	http://127.0.0.1:3200

1. Tujuan Praktikum

Setelah menyelesaikan praktikum ini, praktikan diharapkan mampu:

1. menjelaskan peran runtime, framework, route, controller, service, dan repository;
2. menetapkan resource, input, output, status, dan kondisi gagal pada rancangan endpoint;
3. menyiapkan proyek dan menjalankan API CRUD katalog buku dengan Express;
4. membuktikan keberhasilan serta penolakan input melalui respons aktual; dan
5. menyusun dokumentasi OpenAPI dan mengujinya melalui Swagger UI lokal.

2. Bukti Foto dan Pembahasan

Bagian ini memuat bukti foto hasil pengerjaan beserta penjelasan pada setiap foto. Foto dikelompokkan menurut tahap pengerjaan, mulai dari persiapan lingkungan, pengujian endpoint, dokumentasi OpenAPI, hingga tugas mandiri.

2.1 Persiapan Lingkungan (Checkpoint 1)



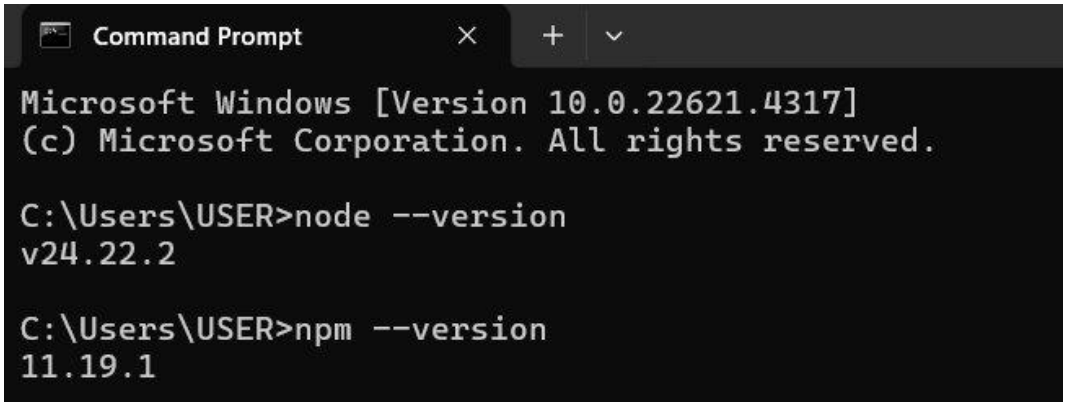
Gambar 1. Pemeriksaan versi Node.js

Perintah `node --version` menampilkan v24.21.0, sehingga Node.js lini 24.x sudah tersedia dan dikenali dari terminal.



Gambar 2. Pemeriksaan versi Node.js dan npm

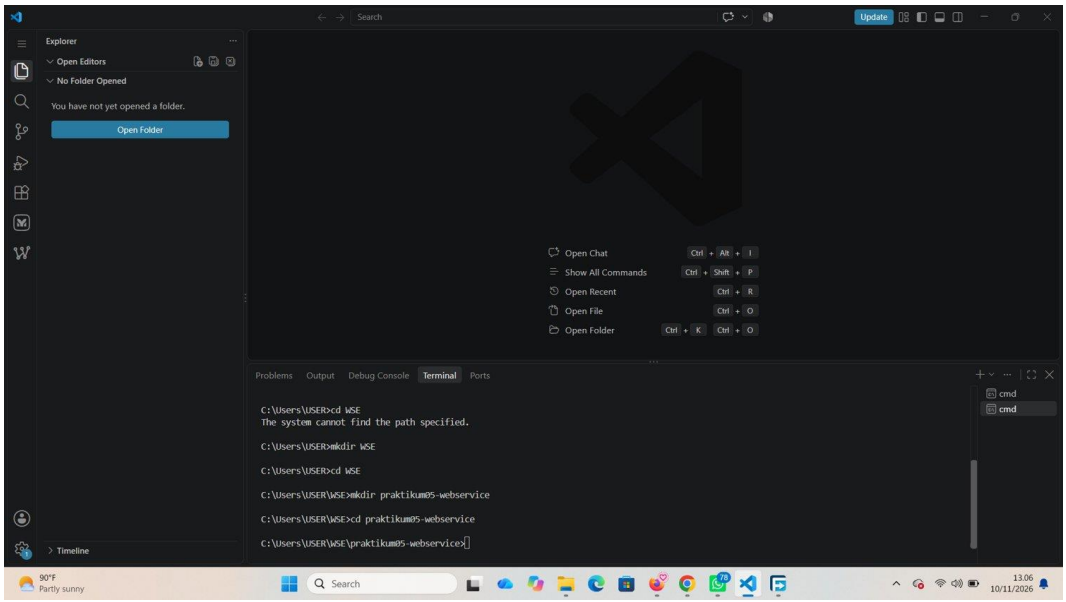
Perintah `npm --version` menampilkan 11.19.0 sehingga npm juga dikenali. Kedua perintah dijalankan pada baris terpisah di Command Prompt.



Gambar 3. Versi setelah pembaruan

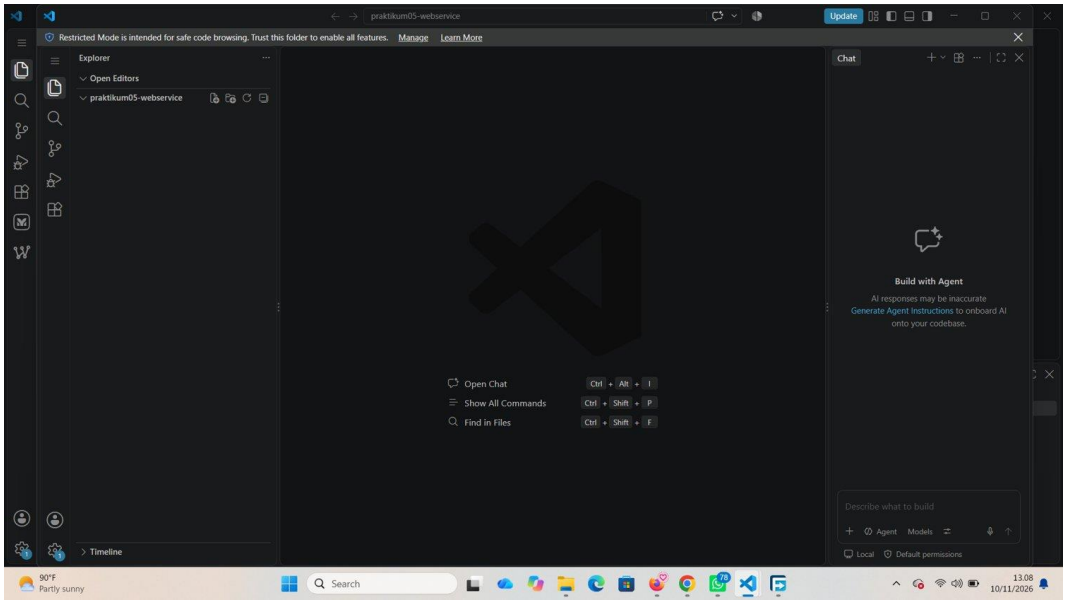
Pemeriksaan ulang menampilkan Node.js v24.22.2 dan npm 11.19.1, masih pada lini 24.x sesuai ketentuan modul. Versi inilah yang dicatat sebagai versi lingkungan akhir.

2.2 Folder Proyek dan Dependensi (Checkpoint 2)



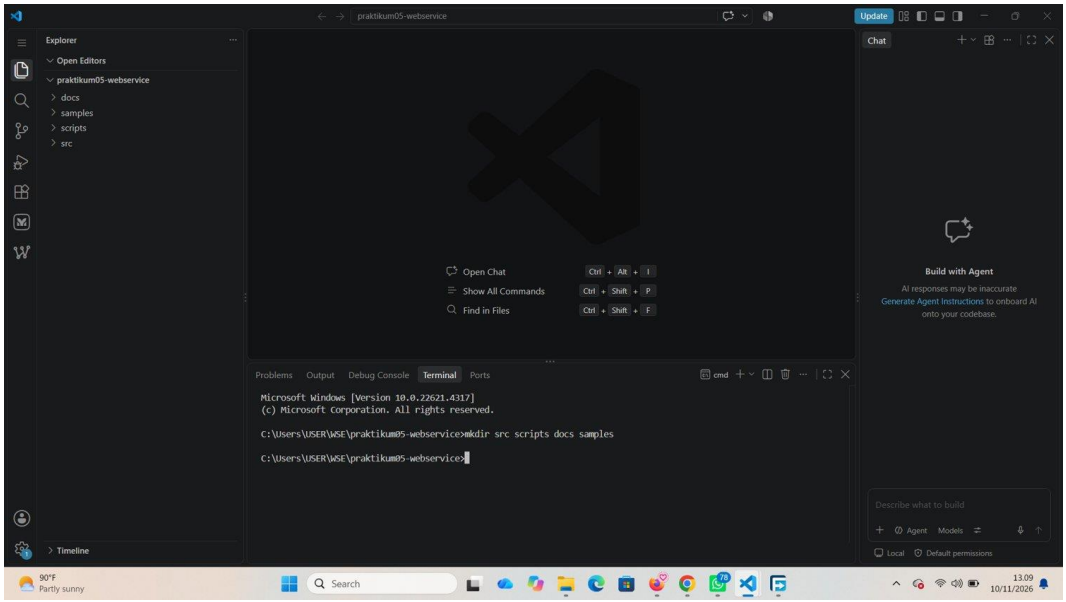
Gambar 4. Membuat folder WSE dan folder proyek

Perintah `cd WSE` pertama menampilkan pesan path tidak ditemukan karena folder belum ada, sehingga folder WSE dibuat dengan `mkdir`. Setelah itu folder praktikum05-webservice dibuat dan dimasuki; lokasi terminal berakhir pada WSE\praktikum05-webservice.



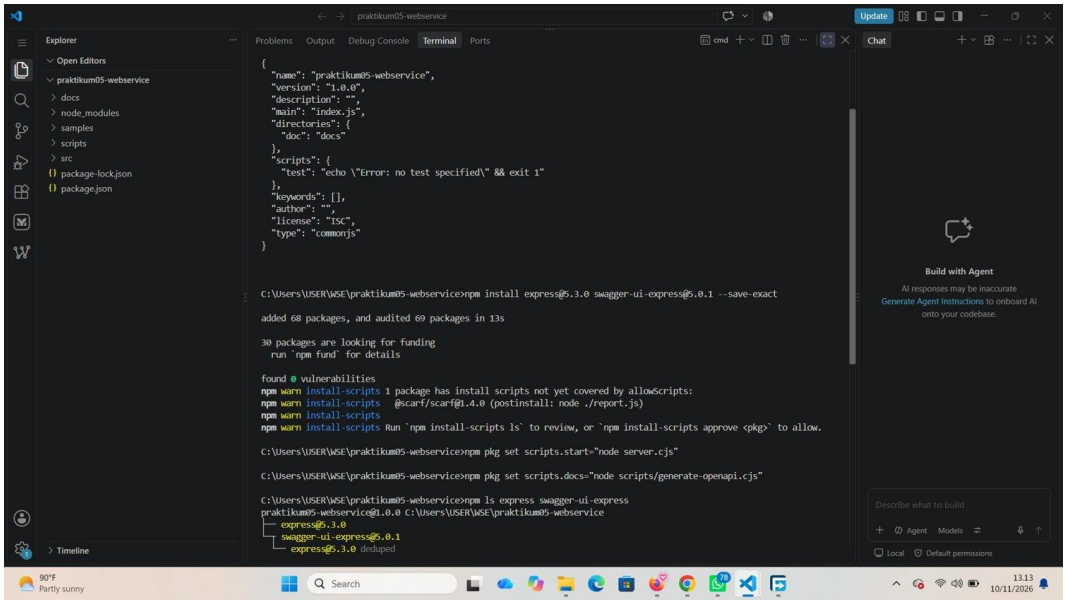
Gambar 5. Proyek dibuka di Visual Studio Code

Folder praktikum05-webservice tampil pada Explorer VS Code. Pemberitahuan Restricted Mode muncul karena folder baru dibuka pada editor.



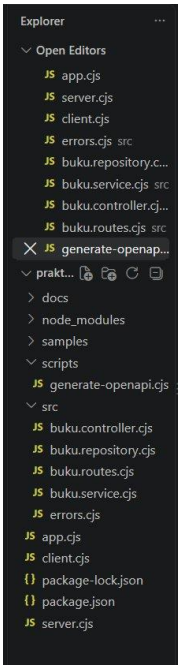
Gambar 6. Membuat subfolder proyek

Perintah `mkdir src scripts docs samples` membuat empat subfolder yang tampil pada Explorer: docs, samples, scripts, dan src.



Gambar 7. Instalasi paket dan pemeriksaan versi

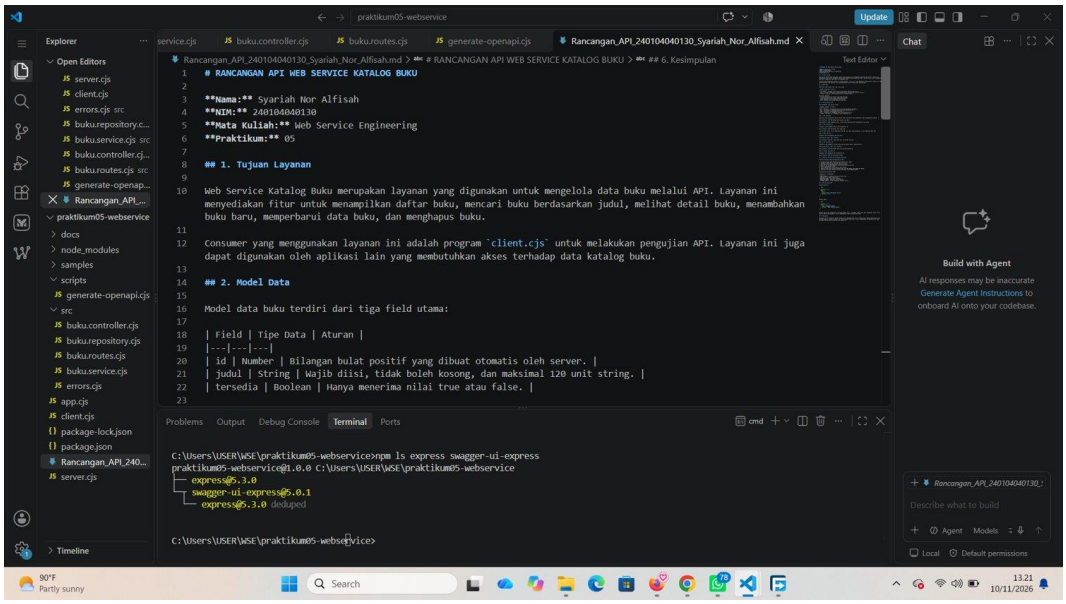
Setelah `npm init -y` membuat package.json, perintah `npm install express@5.3.0 swagger-ui-express@5.0.1 --save-exact` memasang 68 paket tanpa kerentanan. Skrip start dan docs ditambahkan dengan `npm pkg set`, dan `npm ls` memperlihatkan express@5.3.0 serta swagger-ui-express@5.0.1. Pesan install-scripts hanyalah peringatan, bukan error.



Gambar 8. Struktur berkas proyek

Explorer memperlihatkan berkas app.cjs, server.cjs, client.cjs, serta folder src berisi errors, repository, service, controller, dan router sesuai pembagian tanggung jawab pada modul.

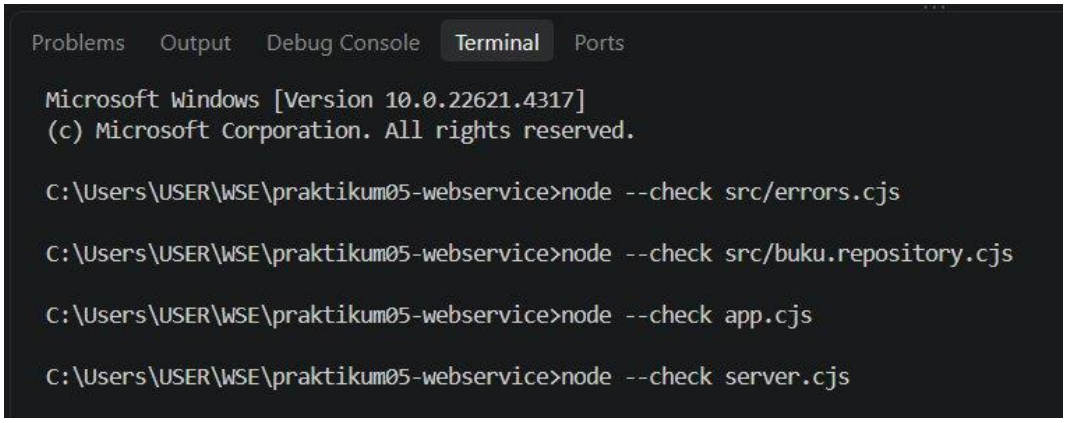
2.3 Rancangan API (Checkpoint 3)



Gambar 9. Catatan Rancangan API

Berkas Rancangan_API_240104040130_Syariah_Nor_Alfisah.md memuat identitas, tujuan layanan (daftar, pencarian, detail, tambah, ubah, hapus), consumer (client.cjs dan aplikasi lain), serta model data id, judul, dan tersedia. Rancangan ini disusun sebelum menulis handler.

2.4 Memeriksa Sintaks dan Menjalankan Server



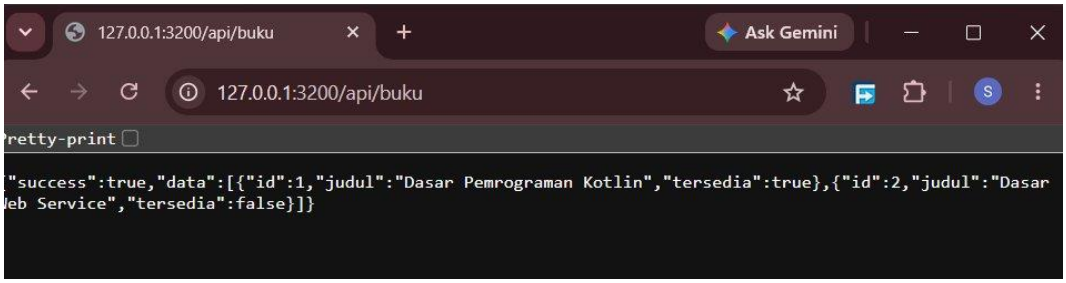
Gambar 10. Pemeriksaan sintaks dengan node --check

Berkas errors.cjs, buku.repository.cjs, app.cjs, dan server.cjs diperiksa dengan `node --check` dan tidak menampilkan pesan error, sehingga sintaks dinyatakan benar.



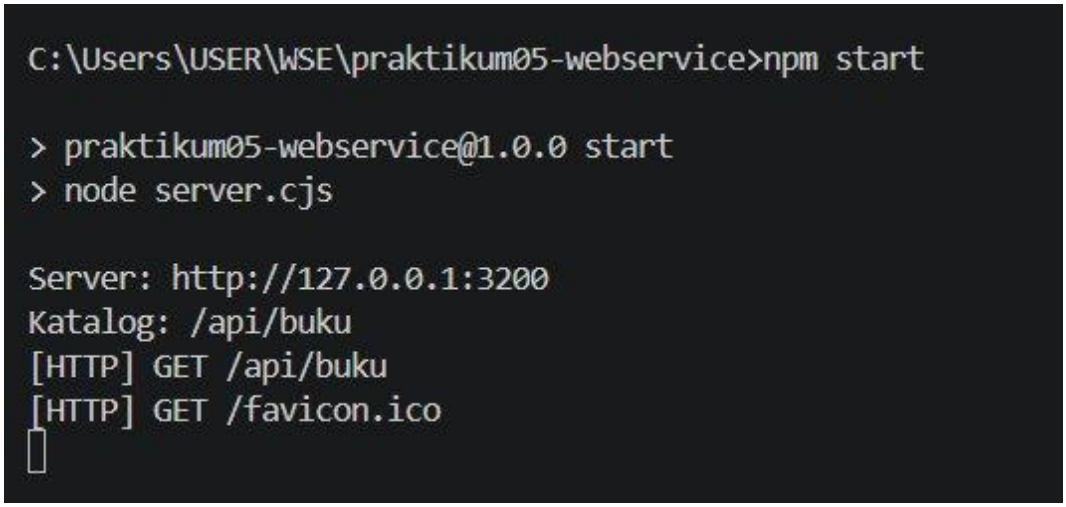
Gambar 11. Server berjalan

`npm start` menjalankan `node server.cjs` dan menampilkan *Server: http://127.0.0.1:3200* serta *Katalog: /api/buku*, yang menandakan listener aktif pada port 3200.



Gambar 12. GET /api/buku pada browser

Alamat 127.0.0.1:3200/api/buku mengembalikan JSON `success:true` dengan dua buku awal: ID 1 *Dasar Pemrograman Kotlin* (tersedia) dan ID 2 *Dasar Web Service* (tidak tersedia).



Gambar 13. Log permintaan pada server

Terminal server mencatat `[HTTP] GET /api/buku` dan `GET /favicon.ico`. Permintaan favicon dikirim otomatis oleh browser sehingga ikut tercatat.

2.5 Membaca dan Mencari Data

```
Problems  Output  Debug Console  Terminal  Ports

Microsoft Windows [Version 10.0.22621.4317]
(c) Microsoft Corporation. All rights reserved.

C:\Users\USER\WSE\praktikum05-webservice>node client.cjs GET /api/buku
Request: GET http://127.0.0.1:3200/api/buku
Body request: (kosong)
HTTP: 200
Content-Type: application/json; charset=utf-8
Location: -
Cache-Control: no-store
Body respons: {"success":true,"data":[{"id":1,"judul":"Dasar Pemrograman Kotlin","tersedia":true},{"id":2,"judul":"Dasar Web S
ervice","tersedia":false}]}
```

Gambar 14. Client: GET /api/buku

Client menampilkan request, HTTP 200, dan body respons berisi array dua buku awal.

```
C:\Users\USER\WSE\praktikum05-webservice>node client.cjs GET /api/buku/1
Request: GET http://127.0.0.1:3200/api/buku/1
Body request: (kosong)
HTTP: 200
Content-Type: application/json; charset=utf-8
Location: -
Cache-Control: no-store
Body respons: {"success":true,"data":{"id":1,"judul":"Dasar Pemrograman Kotlin","tersedia":true}}
```

Gambar 15. Client: GET /api/buku/1

HTTP 200 dengan Content-Type application/json; charset=utf-8 dan Cache-Control no-store; data berupa objek buku ID 1.

```
C:\Users\USER\WSE\praktikum05-webservice>node client.cjs GET /api/buku/2
Request: GET http://127.0.0.1:3200/api/buku/2
Body request: (kosong)
HTTP: 200
Content-Type: application/json; charset=utf-8
Location: -
Cache-Control: no-store
Body respons: {"success":true,"data":{"id":2,"judul":"Dasar Web Service","tersedia":false}}
```

Gambar 16. Client: GET /api/buku/2

HTTP 200; ID 2 *Dasar Web Service* memiliki `tersedia:false`.

```
C:\Users\USER\WSE\praktikum05-webservice>node client.cjs GET "/api/buku?q=kotlin"
Request: GET http://127.0.0.1:3200/api/buku?q=kotlin
Body request: (kosong)
HTTP: 200
Content-Type: application/json; charset=utf-8
Location: -
Cache-Control: no-store
Body respons: {"success":true,"data":[{"id":1,"judul":"Dasar Pemrograman Kotlin","tersedia":true}]}
```

Gambar 17. Pencarian q=kotlin

HTTP 200 dengan satu buku (ID 1) karena judul mengandung kata kotlin.

```
C:\Users\USER\WSE\praktikum05-webservice>node client.cjs GET "/api/buku?q=tidakada"
Request: GET http://127.0.0.1:3200/api/buku?q=tidakada
Body request: (kosong)
HTTP: 200
Content-Type: application/json; charset=utf-8
Location: -
Cache-Control: no-store
Body respons: {"success":true,"data":[]}
```

Gambar 18. Pencarian q=tidakada

HTTP 200 dengan `data: []`. Pencarian tanpa hasil tetap berstatus 200, bukan 404.

2.6 Operasi Tambah, Ubah, dan Hapus

```
C:\Users\USER\WSE\praktikum05-webservice>node client.cjs POST /api/buku samples/buku-baru.json
Request: POST http://127.0.0.1:3200/api/buku
Body request: {
  "judul": "Belajar API dengan Express",
  "tersedia": true
}

HTTP: 201
Content-Type: application/json; charset=utf-8
Location: /api/buku/3
Cache-Control: no-store
Body respons: {"success":true,"data":{"id":3,"judul":"Belajar API dengan Express","tersedia":true}}
```

Gambar 19. POST /api/buku dengan samples/buku-baru.json

Server mengembalikan HTTP 201 dengan header `Location: /api/buku/3`. ID 3 dibuat oleh server dan tidak dikirim pada body.

```
C:\Users\USER\WSE\praktikum05-webservice>node client.cjs GET /api/buku/3
Request: GET http://127.0.0.1:3200/api/buku/3
Body request: (kosong)
HTTP: 200
Content-Type: application/json; charset=utf-8
Location: -
Cache-Control: no-store
Body respons: {"success":true,"data":{"id":3,"judul":"Belajar API dengan Express","tersedia":true}}
```

Gambar 20. GET /api/buku/3 setelah POST

HTTP 200 membuktikan buku baru dapat dibaca pada alamat yang diberikan header Location.

```
C:\Users\USER\WSE\praktikum05-webservice>node client.cjs PUT /api/buku/3 samples/buku-ubah.json
Request: PUT http://127.0.0.1:3200/api/buku/3
Body request: {
  "judul": "Belajar API HTTP",
  "tersedia": false
}

HTTP: 200
Content-Type: application/json; charset=utf-8
Location: -
Cache-Control: no-store
Body respons: {"success":true,"data":{"id":3,"judul":"Belajar API HTTP","tersedia":false}}
```

Gambar 21. PUT /api/buku/3 dengan samples/buku-ubah.json

HTTP 200; ID tetap 3 sedangkan judul berubah menjadi *Belajar API HTTP* dan `tersedia` menjadi false karena PUT mengganti seluruh field yang dapat ditulis.

```
C:\Users\USER\WSE\praktikum05-webservice>node client.cjs GET /api/buku/3
Request: GET http://127.0.0.1:3200/api/buku/3
Body request: (kosong)
HTTP: 200
Content-Type: application/json; charset=utf-8
Location: -
Cache-Control: no-store
Body respons: {"success":true,"data":{"id":3,"judul":"Belajar API HTTP","tersedia":false}}
```

Gambar 22. GET /api/buku/3 setelah PUT

Data yang terbaca sesuai hasil penggantian sehingga perubahan tersimpan pada sesi server.

```
C:\Users\USER\WSE\praktikum05-webservice>node client.cjs DELETE /api/buku/3
Request: DELETE http://127.0.0.1:3200/api/buku/3
Body request: (kosong)
HTTP: 204
Content-Type: null
Location: -
Cache-Control: no-store
Body respons: (kosong)
```

Gambar 23. DELETE /api/buku/3

HTTP 204 dengan Content-Type null dan body kosong. Respons 204 memang tidak memiliki body JSON.

```
C:\Users\USER\WSE\praktikum05-webservice>node client.cjs GET /api/buku/3
Request: GET http://127.0.0.1:3200/api/buku/3
Body request: (kosong)
HTTP: 404
Content-Type: application/json; charset=utf-8
Location: -
Cache-Control: no-store
Body respons: {"success":false,"error":{"code":"BOOK_NOT_FOUND","message":"Buku tidak ditemukan."}}
```

Gambar 24. GET /api/buku/3 setelah DELETE

HTTP 404 dengan kode `BOOK_NOT_FOUND` dan pesan “Buku tidak ditemukan.”, membuktikan buku telah terhapus.

2.7 Pengujian Input Salah dan Error Handling

```
C:\Users\USER\WSE\praktikum05-webservice>node client.cjs GET /api/buku/abc
Request: GET http://127.0.0.1:3200/api/buku/abc
Body request: (kosong)
HTTP: 400
Content-Type: application/json; charset=utf-8
Location: -
Cache-Control: no-store
Body respons: {"success":false,"error":{"code":"INVALID_ID","message":"ID harus berupa bilangan bulat positif."}}
```

Gambar 25. ID abc

HTTP 400 `INVALID_ID`: “ID harus berupa bilangan bulat positif.” ID bukan angka ditolak sebelum mencari data.

```
C:\Users\USER\WSE\praktikum05-webservice>node client.cjs GET /api/buku/1abc
Request: GET http://127.0.0.1:3200/api/buku/1abc
Body request: (kosong)
HTTP: 400
Content-Type: application/json; charset=utf-8
Location: -
Cache-Control: no-store
Body respons: {"success":false,"error":{"code":"INVALID_ID","message":"ID harus berupa bilangan bulat positif."}}
```

Gambar 26. ID 1abc

HTTP 400 `INVALID_ID`. Walaupun `parseInt` dapat menghasilkan angka 1, seluruh teks harus berupa digit sehingga ID ini ditolak.

```
C:\Users\USER\WSE\praktikum05-webservice>node client.cjs GET /api/buku/01
Request: GET http://127.0.0.1:3200/api/buku/01
Body request: (kosong)
HTTP: 400
Content-Type: application/json; charset=utf-8
Location: -
Cache-Control: no-store
Body respons: {"success":false,"error":{"code":"INVALID_ID","message":"ID harus berupa bilangan bulat positif."}}
```

Gambar 27. ID 01

HTTP 400 `INVALID_ID`. ID dengan nol di depan tidak diterima menurut kontrak.

```
C:\Users\USER\WSE\praktikum05-webservice>node client.cjs GET /api/buku/999
Request: GET http://127.0.0.1:3200/api/buku/999
Body request: (kosong)
HTTP: 404
Content-Type: application/json; charset=utf-8
Location: -
Cache-Control: no-store
Body respons: {"success":false,"error":{"code":"BOOK_NOT_FOUND","message":"Buku tidak ditemukan."}}
```

Gambar 28. ID 999

HTTP 404 `BOOK_NOT_FOUND`. ID valid tetapi buku tidak ada, sehingga bedanya dengan 400 adalah pada validitas format ID.

```
C:\Users\USER\WSE\praktikum05-webservice>node client.cjs POST /api/buku samples/buku-salah-tipe.json
Request: POST http://127.0.0.1:3200/api/buku
Body request: {
  "judul": "Belajar API",
  "tersedia": "true"
}

HTTP: 400
Content-Type: application/json; charset=utf-8
Location: -
Cache-Control: no-store
Body respons: {"success":false,"error":{"code":"INVALID_BOOK","message":"Isi hanya judul 1-120 karakter dan tersedia boolean."}}
```

Gambar 29. POST dengan tersedia berupa string

Body `buku-salah-tipe.json` memuat `"tersedia":"true"`; server menjawab HTTP 400 `INVALID_BOOK` karena field harus Boolean.



Gambar 30. PUT parsial

PUT /api/buku/1 hanya dengan judul menghasilkan HTTP 400 `INVALID_BOOK` karena PUT mensyaratkan seluruh field yang dapat ditulis.



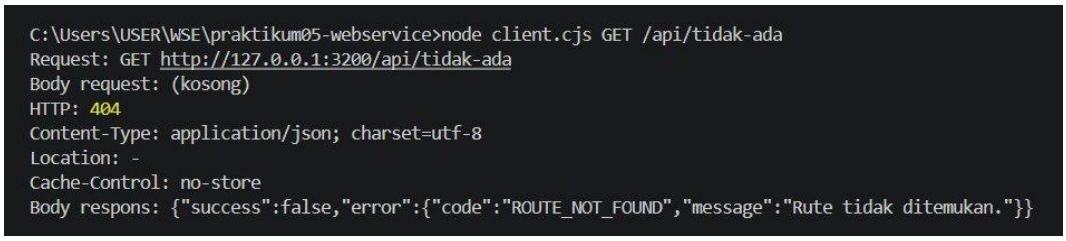
Gambar 31. JSON sengaja rusak

File `json-rusak.txt` berisi JSON tidak lengkap; server menjawab HTTP 400 `INVALID_JSON`: “Body harus berupa objek JSON yang valid.”



Gambar 32. Media type text/plain

POST dengan Content-Type text/plain menghasilkan HTTP 415 `UNSUPPORTED_MEDIA_TYPE`: “Gunakan Content-Type application/json.” walaupun isi body sebenarnya valid.



Gambar 33. Path tidak dikenal

GET /api/tidak-ada menghasilkan HTTP 404 `ROUTE_NOT_FOUND`: “Rute tidak ditemukan.” Kode ini berbeda dari `BOOK_NOT_FOUND` karena yang tidak ada adalah route, bukan buku.

```
C:\Users\USER\WSE\praktikum05-webservice>node client.cjs GET /api/buku/1
Request: GET http://127.0.0.1:3200/api/buku/1
Body request: (kosong)
HTTP: 200
Content-Type: application/json; charset=utf-8
Location: -
Cache-Control: no-store
Body respons: {"success":true,"data":{"id":1,"judul":"Dasar Pemrograman Kotlin","tersedia":true}}
```

Gambar 34. GET /api/buku/1 setelah PUT parsial

Data ID 1 tetap *Dasar Pemrograman Kotlin* dengan `tersedia:true`, membuktikan data target tidak berubah ketika permintaan ditolak.

```
C:\Users\USER\WSE\praktikum05-webservice>node client.cjs GET "/api/buku?q=a&q=b"
Request: GET http://127.0.0.1:3200/api/buku?q=a&q=b
Body request: (kosong)
HTTP: 400
Content-Type: application/json; charset=utf-8
Location: -
Cache-Control: no-store
Body respons: {"success":false,"error":{"code":"INVALID_QUERY","message":"q harus berupa satu teks maksimal 80 karakter."}}
```

Gambar 35. Query q dikirim dua kali

GET `/api/buku?q=a&q=b` menghasilkan HTTP 400 `INVALID_QUERY`: “q harus berupa satu teks maksimal 80 karakter.”

```
Katalog: /api/buku
[HTTP] GET /api/buku
[HTTP] GET /favicon.ico
[HTTP] GET /api/buku
[HTTP] GET /api/buku/1
[HTTP] GET /api/buku/2
[HTTP] GET /api/buku
[HTTP] GET /api/buku
[HTTP] POST /api/buku
[HTTP] GET /api/buku/3
[HTTP] PUT /api/buku/3
[HTTP] GET /api/buku/3
[HTTP] DELETE /api/buku/3
[HTTP] GET /api/buku/3
[HTTP] GET /api/buku/abc
[HTTP] GET /api/buku/1abc
[HTTP] GET /api/buku/01
[HTTP] GET /api/buku/999
[HTTP] POST /api/buku
[HTTP] PUT /api/buku/1
[HTTP] POST /api/buku
[HTTP] POST /api/buku
[HTTP] GET /api/tidak-ada
[HTTP] GET /api/buku/1
[HTTP] GET /api/buku
```

Gambar 36. Log server selama pengujian

Terminal server mencatat setiap request secara berurutan (GET, POST, PUT, DELETE, ID salah, path tidak dikenal, dan seterusnya). Log ini mencocokkan perintah client dengan permintaan yang benar-benar diterima server.

2.8 Dokumentasi OpenAPI dan Swagger UI

```
C:\Users\USER\WSE\praktikum05-webservice>node --check scripts/generate-openapi.cjs

C:\Users\USER\WSE\praktikum05-webservice>npm run docs

> praktikum05-webservice@1.0.0 docs
> node scripts/generate-openapi.cjs

Dokumen dibuat: docs/openapi.json
```

Gambar 37. Generator dokumentasi

Skrip `scripts/generate-openapi.cjs` lolos `node --check`, dan `npm run docs` menampilkan *Dokumen dibuat: docs/openapi.json*. Berkas dokumentasi dihasilkan otomatis, bukan diketik manual.

app.cjs | server.cjs | client.cjs | buku-baru.json | buku-ubah.json | buku-salah-tipe.json

app.cjs > ...

```
9   app.use((req, res, next) => {
10    console.log(`[HTTP] ${req.method} ${req.path}`);
11    res.set('Cache-Control', 'no-store');
12    next();
13  });
14
15  app.use(express.json({ limit: '10kb' }));
16
17  app.use('/api/buku', bukuRoutes);
18
19  const path = require('node:path');
20  const swaggerUi = require('swagger-ui-express');
21
22  const specPath = path.join(__dirname, 'docs', 'openapi.json');
23
24  app.get('/openapi.json', (req, res) => res.sendFile(specPath));
25
26  app.use('/docs', swaggerUi.serve, swaggerUi.setup(null, {
27    swaggerOptions: {
28      url: '/openapi.json',
29      validatorUrl: null
30    }
31  }));
32
```

Gambar 38. Integrasi dokumentasi pada app.cjs

app.cjs memuat middleware log dan `Cache-Control: no-store`, `express.json({ limit: '10kb' })`, router `/api/buku`, route `/openapi.json` yang mengirim docs/openapi.json, serta `/docs` dari swagger-ui-express dengan url `/openapi.json` dan `validatorUrl: null`. Blok dokumentasi diletakkan setelah router katalog.

```
C:\Users\USER\WSE\praktikum05-webservice>node --check app.cjs

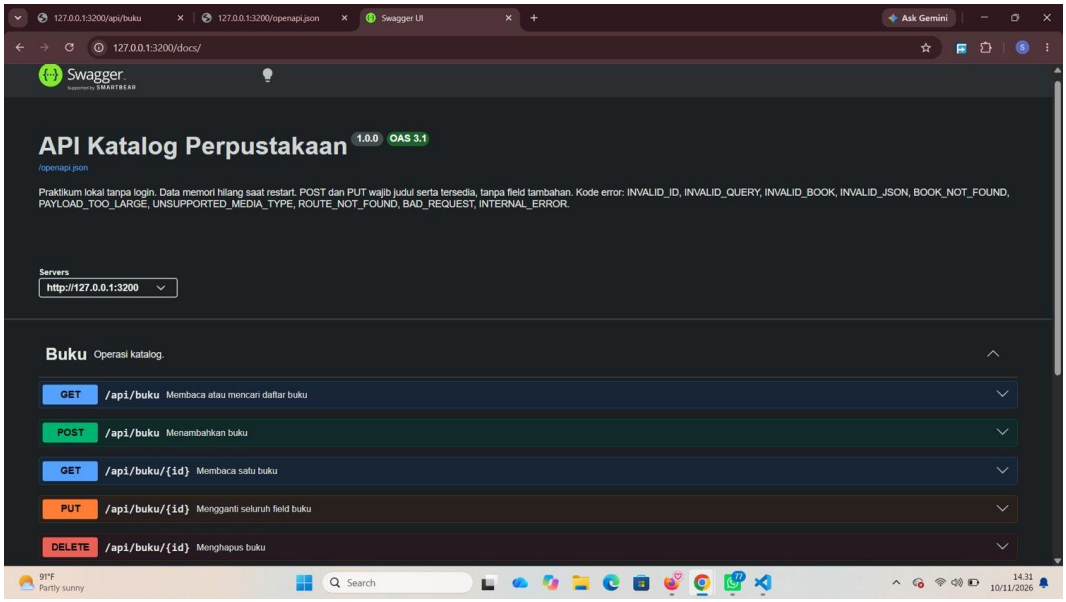
C:\Users\USER\WSE\praktikum05-webservice>npm start

> praktikum05-webservice@1.0.0 start
> node server.cjs

Server: http://127.0.0.1:3200
Katalog: /api/buku
```

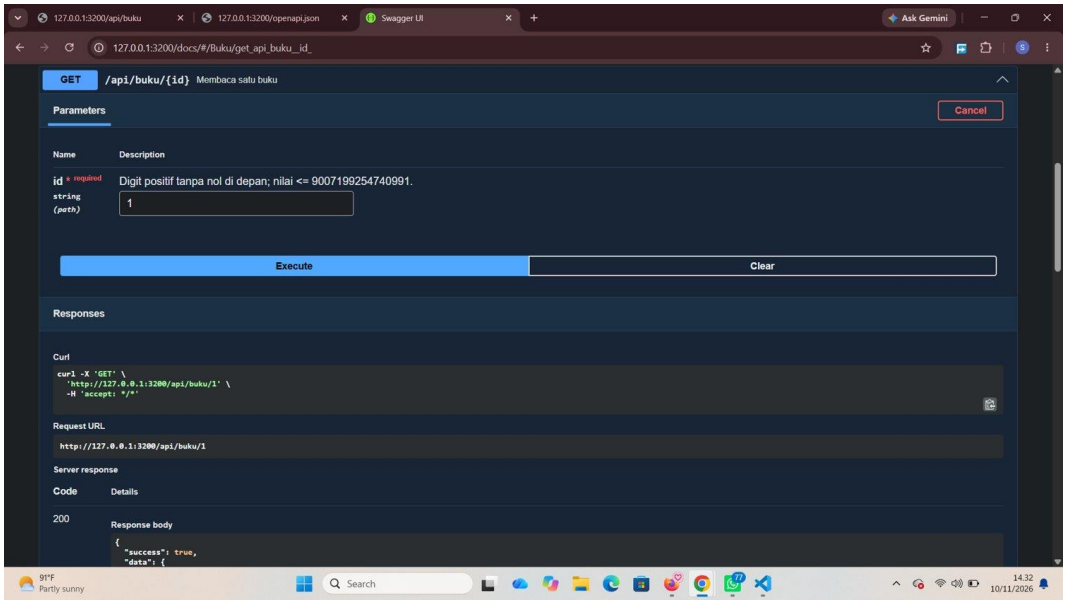
Gambar 39. Server dijalankan ulang dengan dokumentasi

Setelah `node --check app.cjs` berhasil, `npm start` dijalankan kembali agar dokumentasi termuat; server kembali aktif pada http://127.0.0.1:3200.



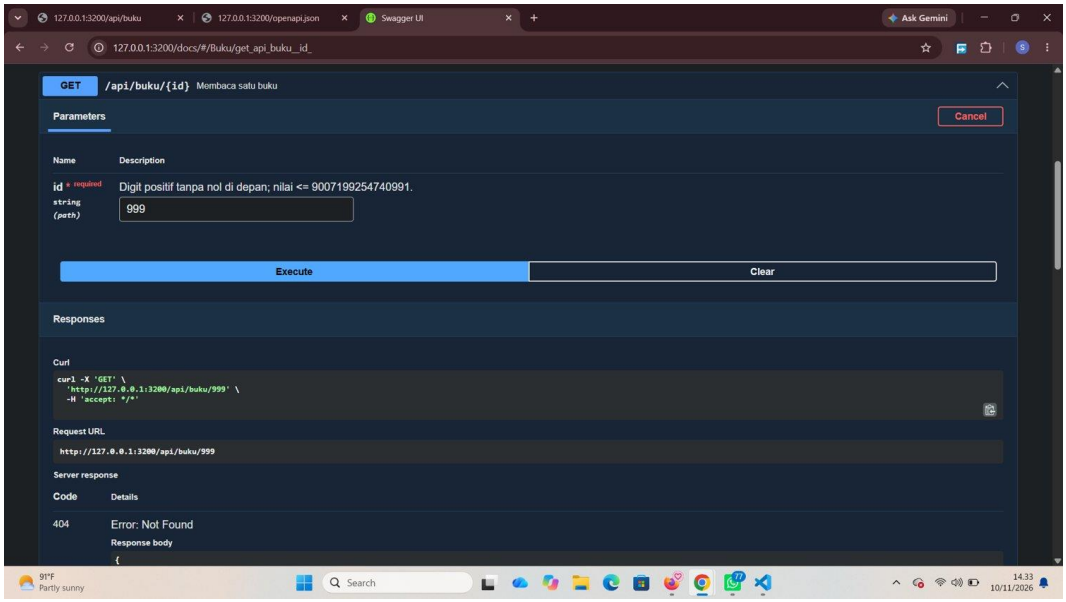
Gambar 40. Halaman Swagger UI /docs/

Halaman memuat judul *API Katalog Perpustakaan* (OAS 3.1), server http://127.0.0.1:3200, dan lima operasi: GET /api/buku, POST /api/buku, GET /api/buku/{id}, PUT /api/buku/{id}, dan DELETE /api/buku/{id}.



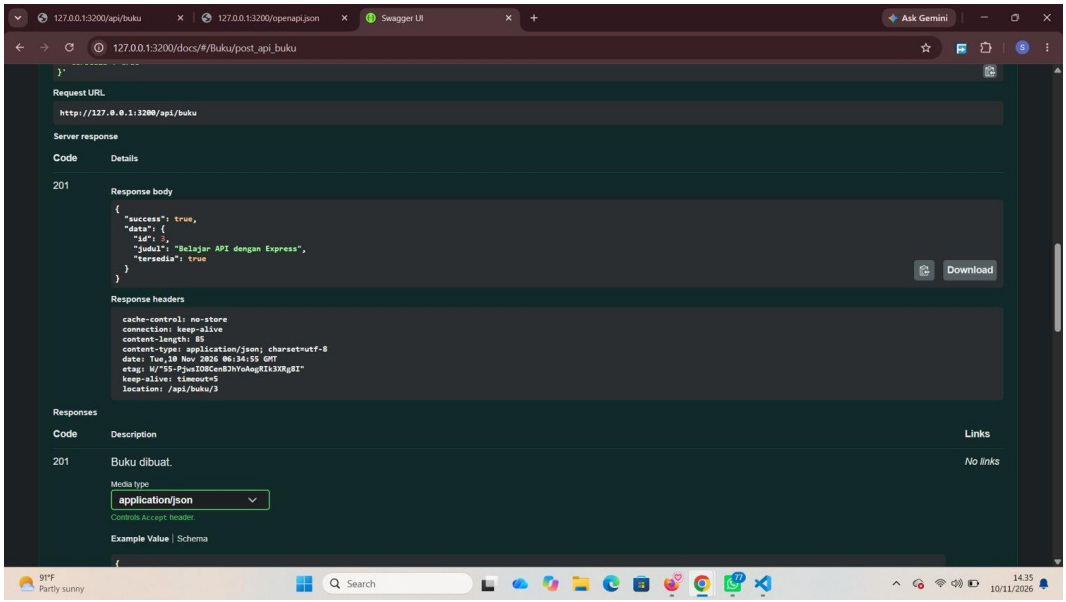
Gambar 41. Swagger UI: GET /api/buku/{id} dengan ID 1

Pada Try it out, id diisi 1 lalu Execute. Request URL http://127.0.0.1:3200/api/buku/1 menghasilkan kode 200 dengan `success:true`.



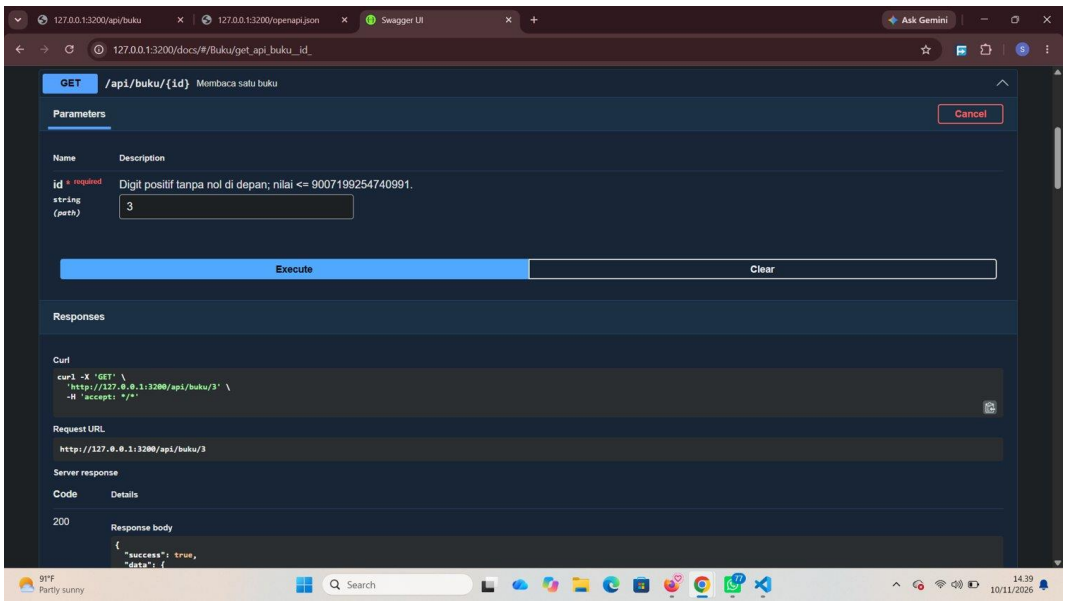
Gambar 42. Swagger UI: GET dengan ID 999

ID 999 menghasilkan kode 404 *Not Found*, sesuai kontrak `BOOK_NOT_FOUND`.



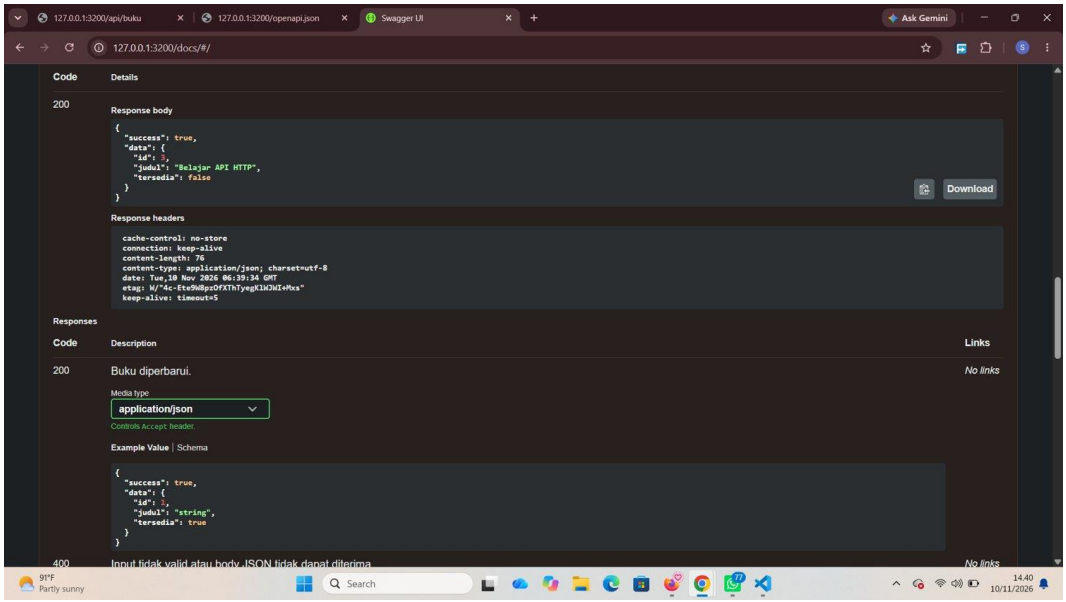
Gambar 43. Swagger UI: POST /api/buku

Execute pada POST menghasilkan kode 201. Body respons memuat ID 3, judul *Belajar API dengan Express*, dan `tersedia:true`, sedangkan header respons memuat `location: /api/buku/3`.



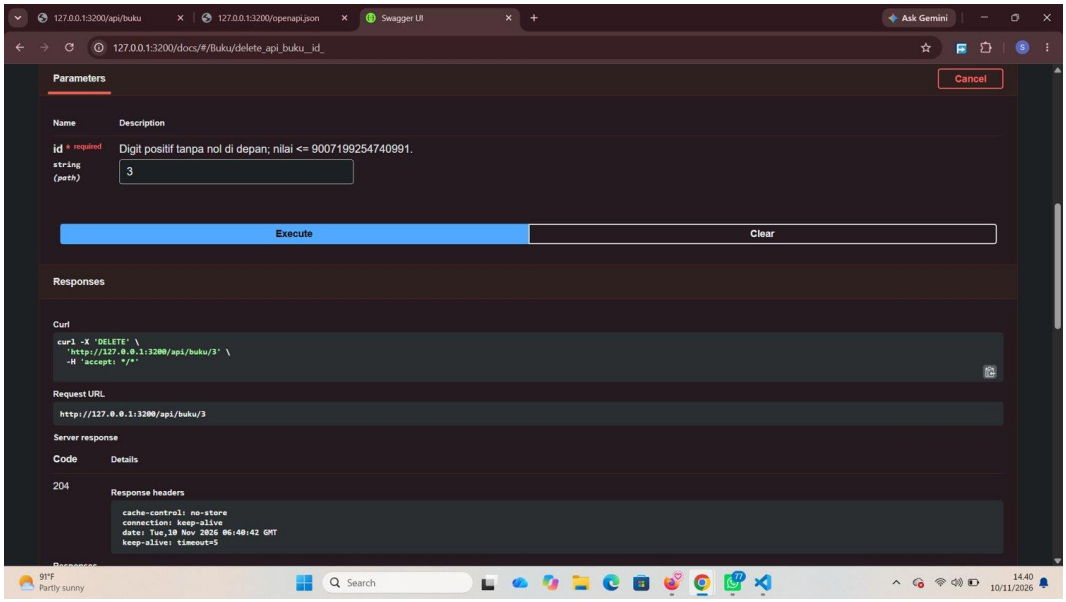
Gambar 44. Swagger UI: GET detail dengan ID 3

GET ID 3 menghasilkan kode 200, membuktikan buku hasil POST dari Swagger UI tersimpan pada server.



Gambar 45. Swagger UI: PUT /api/buku/{id}

PUT menghasilkan kode 200 (*Buku diperbarui*) dengan judul *Belajar API HTTP* dan `tersedia:false`.



Gambar 46. Swagger UI: DELETE /api/buku/{id}

DELETE ID 3 dengan perintah curl `-X DELETE` menghasilkan kode 204 tanpa body, hanya header respons.



Gambar 47. Bagian Schemas pada Swagger UI

Bagian Schemas menampilkan BukuInput, Buku, BukuResponse, DaftarResponse, dan ErrorResponse. Ini menunjukkan dokumen OpenAPI memuat model data dan bentuk respons selain daftar operasi.

```
> praktikum05-webservice@1.0.0 start
> node server.cjs

Server: http://127.0.0.1:3200
Katalog: /api/buku
[HTTP] GET /openapi.json
[HTTP] GET /docs
[HTTP] GET /docs/
[HTTP] GET /docs/swagger-ui.css
[HTTP] GET /docs/swagger-ui-bundle.js
[HTTP] GET /docs/swagger-ui-standalone-preset.js
[HTTP] GET /docs/swagger-ui-init.js
[HTTP] GET /openapi.json
[HTTP] GET /docs/favicon-32x32.png
[HTTP] GET /api/buku/1
[HTTP] GET /api/buku/999
[HTTP] POST /api/buku
[HTTP] POST /api/buku
[HTTP] GET /api/buku/3
[HTTP] PUT /api/buku/3
[HTTP] DELETE /api/buku/3
[HTTP] POST /api/buku
[HTTP] GET /api/buku/4
```

Gambar 48. Log server selama memakai Swagger UI

Log mencatat permintaan /openapi.json, /docs/, serta aset swagger-ui, kemudian GET /api/buku/1, GET /api/buku/999, POST, GET /api/buku/3, PUT, dan DELETE. Hal ini membuktikan Execute pada Swagger UI mengirim request nyata ke API, bukan sekadar menampilkan contoh. Log memperlihatkan dua permintaan POST /api/buku selama sesi ini.

2.9 Batas Penyimpanan Dalam Memori

```
C:\Users\USER\WSE\praktikum05-webservice>node client.cjs POST /api/buku samples/buku-baru.json
Request: POST http://127.0.0.1:3200/api/buku
Body request: {
  "judul": "Belajar API dengan Express",
  "tersedia": true
}

HTTP: 201
Content-Type: application/json; charset=utf-8
Location: /api/buku/5
Cache-Control: no-store
Body respons: {"success":true,"data":{"id":5,"judul":"Belajar API dengan Express","tersedia":true}}
```

Gambar 49. POST tambahan melalui client

POST valid menghasilkan HTTP 201 dengan ID 5 dan `Location: /api/buku/5`. ID 3 dan 4 sudah terpakai pada sesi server yang sama, dan ID tidak dipakai ulang setelah penghapusan.

```
C:\Users\USER\WSE\praktikum05-webservice>node client.cjs GET /api/buku/4
Request: GET http://127.0.0.1:3200/api/buku/4
Body request: (kosong)
HTTP: 200
Content-Type: application/json; charset=utf-8
Location: -
Cache-Control: no-store
Body respons: {"success":true,"data":{"id":4,"judul":"Belajar API dengan Express","tersedia":true}}
```

Gambar 50. GET /api/buku/4 saat server berjalan

Buku dengan ID 4 masih terbaca (HTTP 200) selama server berjalan, sehingga data tambahan ada di memori proses yang sedang aktif.

```
C:\Users\USER\WSE\praktikum05-webservice>node client.cjs GET /api/buku/1
Request: GET http://127.0.0.1:3200/api/buku/1
Body request: (kosong)
Tidak memperoleh respons HTTP lengkap: TypeError
```

Gambar 51. Server dihentikan

Ketika server mati, `node client.cjs GET /api/buku/1` menampilkan *Tidak memperoleh respons HTTP lengkap: TypeError*. Tidak ada status HTTP sama sekali, sehingga kondisi ini bukan 404 atau 500 melainkan kegagalan koneksi.

```
C:\Users\USER\WSE\praktikum05-webservice>node client.cjs GET /api/buku
Request: GET http://127.0.0.1:3200/api/buku
Body request: (kosong)
HTTP: 200
Content-Type: application/json; charset=utf-8
Location: -
Cache-Control: no-store
Body respons: {"success":true,"data":[{"id":1,"judul":"Dasar Pemrograman Kotlin","tersedia":true},{"id":2,"judul":"Dasar Web Serv
ice","tersedia":false}]}
```

Gambar 52. GET /api/buku setelah server dimulai ulang

HTTP 200 dengan array hanya dua buku awal (ID 1 dan 2): data tambahan tidak ada lagi.

```
C:\Users\USER\WSE\praktikum05-webservice>node client.cjs GET /api/buku/4
Request: GET http://127.0.0.1:3200/api/buku/4
Body request: (kosong)
HTTP: 404
Content-Type: application/json; charset=utf-8
Location: -
Cache-Control: no-store
Body respons: {"success":false,"error":{"code":"BOOK_NOT_FOUND","message":"Buku tidak ditemukan."}}
```

Gambar 53. GET /api/buku/4 setelah restart

HTTP 404 `BOOK_NOT_FOUND` untuk ID 4. Buku tambahan hilang karena array diinisiasi ulang saat proses dimulai, yang membuktikan batas penyimpanan dalam memori.

```
C:\Users\USER\WSE\praktikum05-webservice>npm start

> praktikum05-webservice@1.0.0 start
> node server.cjs

Server: http://127.0.0.1:3200
Katalog: /api/buku
[HTTP] GET /api/buku
[HTTP] GET /api/buku/4
```

Gambar 54. Log server setelah restart

Server yang baru dijalankan mencatat `GET /api/buku` dan `GET /api/buku/4` dari dua pengujian sebelumnya.

2.10 Keterulangan Proyek

```
Terminate batch job (Y/N)? Y
```

Gambar 55. Server dihentikan sebelum pemasangan ulang

Server dihentikan dengan Ctrl+C dan konfirmasi *Terminate batch job (Y/N)? Y* sebelum `npm ci` dijalankan.

```
C:\Users\USER\WSE\praktikum05-webservice>npm ci

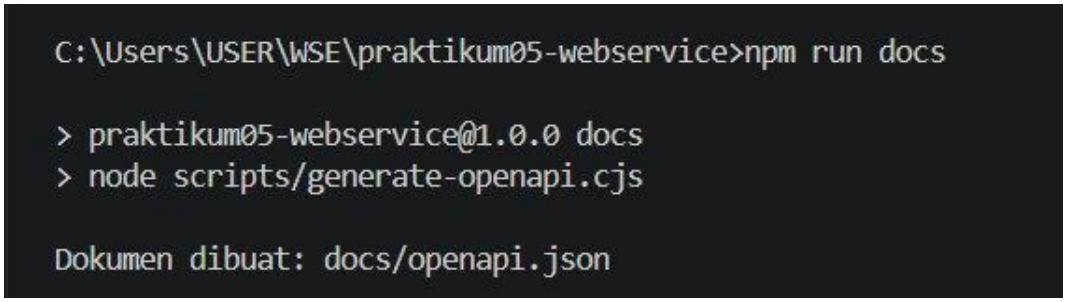
added 68 packages, and audited 69 packages in 11s

30 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
npm warn install-scripts 1 package has install scripts not yet covered by allowScripts:
npm warn install-scripts   @scarf/scarf@1.4.0 (postinstall: node ./report.js)
npm warn install-scripts
npm warn install-scripts Run `npm install-scripts ls` to review, or `npm install-scripts approve <pkg>` to allow.
```

Gambar 56. npm ci

`npm ci` memasang ulang dependensi sesuai lockfile: 68 paket ditambahkan, 69 paket diaudit dalam 11 detik, dan tidak ada kerentanan. Peringatan install-scripts untuk @scarf/scarf@1.4.0 hanya berupa peringatan.



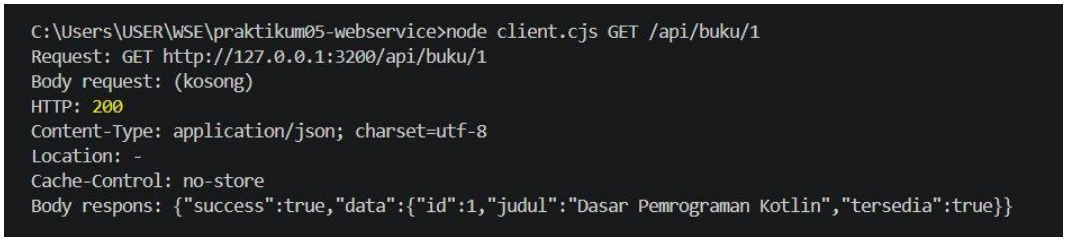
Gambar 57. npm run docs

Generator dijalankan kembali dan menampilkan *Dokumen dibuat: docs/openapi.json*, sehingga dokumentasi dapat dibuat ulang dari generator.



Gambar 58. npm start setelah instalasi ulang

Server kembali berjalan pada http://127.0.0.1:3200 dengan katalog /api/buku.



Gambar 59. GET /api/buku/1 setelah instalasi ulang

HTTP 200 dengan data ID 1 *Dasar Pemrograman Kotlin*, menegaskan proyek dapat dijalankan ulang dari awal sesuai petunjuk.

2.11 Tugas Mandiri 1: Menambah Data Melalui API

```
C:\Users\USER\WSE\praktikum05-webservice>node client.cjs POST /api/buku samples/buku-baru.json
Request: POST http://127.0.0.1:3200/api/buku
Body request: {
  "judul": "Belajar API dengan Express",
  "tersedia": true
}

HTTP: 201
Content-Type: application/json; charset=utf-8
Location: /api/buku/3
Cache-Control: no-store
Body respons: {"success":true,"data":{"id":3,"judul":"Belajar API dengan Express","tersedia":true}}
```

Gambar 60. POST buku pertama

POST `samples/buku-baru.json` menghasilkan HTTP 201; ID aktual 3 dan `Location: /api/buku/3` (data awal kembali dua buku setelah restart).

```
C:\Users\USER\WSE\praktikum05-webservice>node client.cjs POST /api/buku samples/buku-kedua.json
Request: POST http://127.0.0.1:3200/api/buku
Body request: {
  "judul": "Pemrograman JavaScript Dasar",
  "tersedia": true
}

HTTP: 201
Content-Type: application/json; charset=utf-8
Location: /api/buku/4
Cache-Control: no-store
Body respons: {"success":true,"data":{"id":4,"judul":"Pemrograman JavaScript Dasar","tersedia":true}}
```

Gambar 61. POST buku kedua

POST `samples/buku-kedua.json` menambahkan *Pemrograman JavaScript Dasar* dengan ID aktual 4 (HTTP 201), sehingga dua buku baru telah ditambahkan.

```
C:\Users\USER\WSE\praktikum05-webservice>node client.cjs GET /api/buku/3
Request: GET http://127.0.0.1:3200/api/buku/3
Body request: (kosong)
HTTP: 200
Content-Type: application/json; charset=utf-8
Location: -
Cache-Control: no-store
Body respons: {"success":true,"data":{"id":3,"judul":"Belajar API dengan Express","tersedia":true}}
```

Gambar 62. GET /api/buku/3

Buku ID 3 terbaca dengan HTTP 200.

```
C:\Users\USER\WSE\praktikum05-webservice>node client.cjs GET /api/buku/4
Request: GET http://127.0.0.1:3200/api/buku/4
Body request: (kosong)
HTTP: 200
Content-Type: application/json; charset=utf-8
Location: -
Cache-Control: no-store
Body respons: {"success":true,"data":{"id":4,"judul":"Pemrograman JavaScript Dasar","tersedia":true}}
```

Gambar 63. GET /api/buku/4

Buku ID 4 terbaca dengan HTTP 200.

```
C:\Users\USER\WSE\praktikum05-webservice>node client.cjs PUT /api/buku/3 samples/buku-ubah.jso
Request: PUT http://127.0.0.1:3200/api/buku/3
Body request: {
  "judul": "Belajar API HTTP",
  "tersedia": false
}

HTTP: 200
Content-Type: application/json; charset=utf-8
Location: -
Cache-Control: no-store
Body respons: {"success":true,"data":{"id":3,"judul":"Belajar API HTTP","tersedia":false}}
```

Gambar 64. PUT /api/buku/3

PUT dengan `samples/buku-ubah.json` menghasilkan HTTP 200; judul menjadi *Belajar API HTTP* dan `tersedia` false, sedangkan ID tetap 3.

```
C:\Users\USER\WSE\praktikum05-webservice>node client.cjs GET /api/buku/3
Request: GET http://127.0.0.1:3200/api/buku/3
Body request: (kosong)
HTTP: 200
Content-Type: application/json; charset=utf-8
Location: -
Cache-Control: no-store
Body respons: {"success":true,"data":{"id":3,"judul":"Belajar API HTTP","tersedia":false}}
```

Gambar 65. GET /api/buku/3 setelah PUT

Data yang terbaca sesuai perubahan sehingga operasi ubah terbukti.

```
C:\Users\USER\WSE\praktikum05-webservice>node client.cjs DELETE /api/buku/4
Request: DELETE http://127.0.0.1:3200/api/buku/4
Body request: (kosong)
HTTP: 204
Content-Type: null
Location: -
Cache-Control: no-store
Body respons: (kosong)
```

Gambar 66. DELETE /api/buku/4

HTTP 204 dengan body kosong; buku ID 4 dihapus.

```
C:\Users\USER\WSE\praktikum05-webservice>node client.cjs GET /api/buku/4
Request: GET http://127.0.0.1:3200/api/buku/4
Body request: (kosong)
HTTP: 404
Content-Type: application/json; charset=utf-8
Location: -
Cache-Control: no-store
Body respons: {"success":false,"error":{"code":"BOOK_NOT_FOUND","message":"Buku tidak ditemukan."}}
```

Gambar 67. GET /api/buku/4 setelah DELETE

HTTP 404 membuktikan buku ID 4 sudah tidak ada.

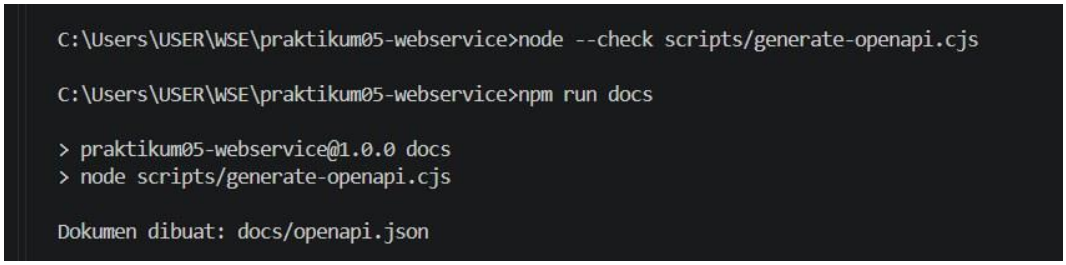
Dua buku ditambahkan melalui POST (ID aktual 3 *Belajar API dengan Express* dan ID 4 *Pemrograman JavaScript Dasar*), keduanya dibaca (Gambar 62 dan Gambar 63), buku ID 3 diubah menjadi *Belajar API HTTP* dengan `tersedia`

false (Gambar 64), dan buku ID 4 dihapus dengan hasil 404 pada pembacaan berikutnya (Gambar 66 dan Gambar 67).

Menyunting array pada berkas berbeda dengan operasi melalui API. Penyuntingan berkas dilakukan langsung pada penyimpanan oleh pengembang, melewati validasi, pembuatan ID oleh server, dan aturan kontrak, serta baru berlaku setelah server dimulai ulang. Operasi melalui API dikirim consumer melalui HTTP, diperiksa oleh router, controller, dan service, dan hasilnya dilaporkan dengan status serta kode kesalahan yang jelas.

2.12 Tugas Mandiri 2: Melengkapi Dokumentasi dan Bukti

Dokumentasi OpenAPI dilengkapi pada generator `scripts/generate-openapi.cjs` dengan deskripsi API, deskripsi operasi, dan contoh respons kesalahan, lalu dokumen dibuat ulang dengan `npm run docs` (Gambar 68). Percobaan `npm start` setelah perubahan sempat gagal dengan pesan EADDRINUSE (Gambar 69) karena server sebelumnya masih berjalan pada port 3200. Setelah server lama dihentikan dan dijalankan ulang, Swagger UI menampilkan dokumentasi yang telah diperbarui (Gambar 70 dan Gambar 71). Ringkasan perubahan ada pada Tabel 1.



Gambar 68. Generator dijalankan ulang: npm run docs

Skrip generator lolos `node --check`, kemudian `npm run docs` menjalankan `node scripts/generate-openapi.cjs` dan menampilkan *Dokumen dibuat: docs/openapi.json*. Dokumen OpenAPI dibuat ulang dari generator setelah deskripsi dan contoh respons kesalahan ditambahkan.

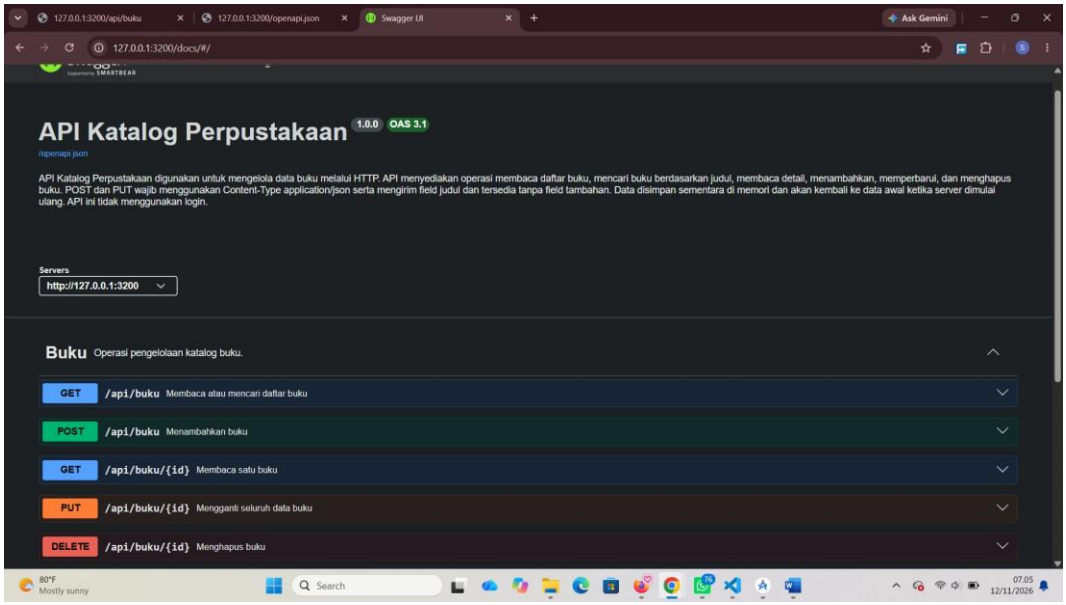
```
C:\Users\USER\WSE\praktikum05-webservice>npm start

> praktikum05-webservice@1.0.0 start
> node server.cjs

Server: http://127.0.0.1:3200
Katalog: /api/buku
Server gagal: EADDRINUSE listen EADDRINUSE: address already in use 127.0.0.1:3200
```

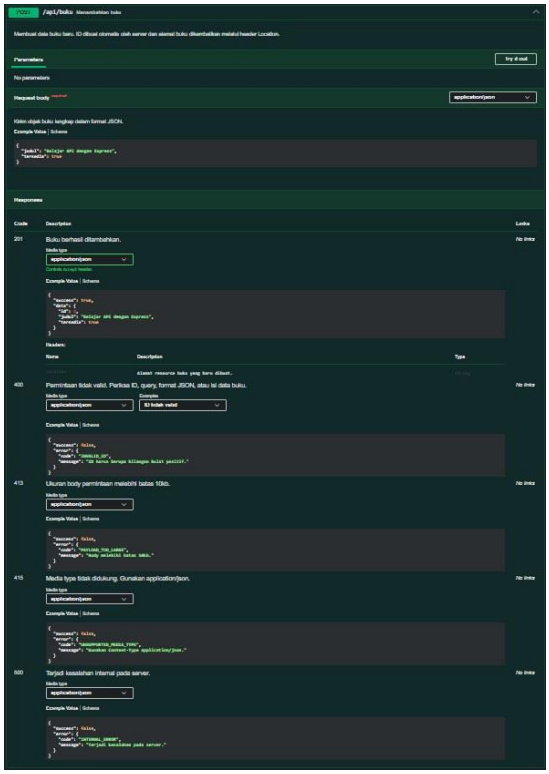
Gambar 69. npm start gagal: port 3200 sudah dipakai

`npm start` menampilkan *Server gagal: EADDRINUSE listen EADDRINUSE: address already in use 127.0.0.1:3200*. Artinya port 3200 masih dipakai server sebelumnya sehingga proses baru tidak dapat berjalan, dan dokumentasi baru belum terlihat sebelum server lama dihentikan lalu dijalankan ulang.



Gambar 70. Swagger UI dengan deskripsi API yang diperbarui

Halaman /docs/ menampilkan deskripsi API yang lebih lengkap: API dipakai untuk mengelola data buku melalui HTTP (daftar, pencarian, detail, tambah, ubah, hapus); POST dan PUT wajib memakai Content-Type application/json dengan field judul dan tersedia tanpa field tambahan; data disimpan sementara di memori dan kembali ke data awal ketika server dimulai ulang; serta API tidak menggunakan login. Server http://127.0.0.1:3200 dan lima operasi tetap tampil.



Gambar 71. Detail operasi POST /api/buku setelah dokumentasi dilengkapi

Deskripsi operasi menyebutkan bahwa ID dibuat otomatis oleh server dan alamat buku dikembalikan melalui header Location. Request body application/json disertai contoh judul dan tersedia. Respons yang didokumentasikan adalah 201 (buku berhasil ditambahkan, dengan header Location), 400 (permintaan tidak valid, contoh INVALID_ID), 413 (body melebihi 10kb, PAYLOAD_TOO_LARGE), 415 (UNSUPPORTED_MEDIA_TYPE), dan 500 (INTERNAL_ERROR), masing-masing dengan contoh body kesalahan.

Tabel 1. Perubahan dokumentasi OpenAPI pada Tugas 2

Bagian	Perubahan	Bukti
Deskripsi API	Menjelaskan fungsi API, kewajiban application/json dengan field judul dan tersedia, penyimpanan sementara di memori, dan tanpa login.	Gambar 70
Deskripsi operasi POST	Menjelaskan ID dibuat server dan alamat buku dikembalikan melalui header Location.	Gambar 71
Contoh respons berhasil	201 dengan contoh body dan keterangan header Location.	Gambar 71
Contoh respons kesalahan	400 (INVALID_ID), 413 (PAYLOAD_TOO_LARGE), 415 (UNSUPPORTED_MEDIA_TYPE), dan 500 (INTERNAL_ERROR).	Gambar 71
Pembuatan ulang dokumen	`npm run docs` menghasilkan docs/openapi.json; server dijalankan ulang agar dokumen termuat.	Gambar 68, Gambar 69

2.13 Tugas Mandiri 3: Pertanyaan Pemahaman

[1] Apa perbedaan Node.js dan Express dalam proyek ini?

Node.js adalah runtime yang menjalankan JavaScript di sisi server, sedangkan Express adalah framework di atas Node.js yang menyediakan routing, middleware, dan penanganan request/response sehingga kode API lebih ringkas.

[2] Mengapa controller dipisahkan dari service dan repository?

Agar tanggung jawab jelas: controller menangani konteks HTTP (status, header, body), service memeriksa kebutuhan aplikasi, dan repository mengakses data. Perubahan pada satu lapisan, misalnya mengganti array dengan database, tidak harus mengubah lapisan lain.

[3] Apa perbedaan parameter path, query, header, dan body?

Path mengidentifikasi resource (`/api/buku/1`), query menyaring atau mengatur permintaan (`?q=kotlin`), header membawa metadata seperti Content-Type, dan body membawa data utama pada POST atau PUT.

[4] Mengapa JSON yang sah belum tentu memenuhi kontrak objek buku?

JSON yang sah hanya berarti sintaksnya benar. Kontrak menuntut field, tipe, dan batas tertentu; misalnya `tersedia` berupa string atau field judul hilang tetap ditolak dengan 400 `INVALID_BOOK`.

[5] Mengapa ID 1abc ditolak walaupun parseInt dapat menghasilkan angka?

`parseInt("1abc")` menghasilkan 1 dan mengabaikan sisa teks. Kontrak mensyaratkan seluruh ID berupa digit desimal positif tanpa nol di depan, sehingga ID harus diperiksa utuh dan ditolak dengan 400 `INVALID_ID`.

[6] Mengapa PUT parsial ditolak pada kontrak ini?

PUT pada kontrak ini mengganti seluruh field yang dapat ditulis. Body yang hanya berisi judul tidak lengkap sehingga ditolak 400 dan data target tidak berubah; PATCH belum disediakan.

[7] Mengapa respons DELETE 204 tidak dibaca sebagai JSON?

Status 204 berarti berhasil tanpa body, sehingga tidak ada JSON untuk diurai. Client cukup membaca status; memaksa parsing JSON akan menimbulkan kesalahan.

[8] Apa perbedaan OpenAPI, Swagger UI, dan implementasi Express?

OpenAPI adalah dokumen deskripsi API, Swagger UI adalah antarmuka yang membaca dokumen itu dan dapat mengirim request, sedangkan implementasi Express adalah kode yang benar-benar menjalankan layanan. Dokumen harus sesuai dengan implementasi.

[9] Mengapa Example Value bukan bukti bahwa endpoint berhasil?

Example Value hanya contoh dari dokumen. Bukti berhasil adalah Server response setelah Execute, yaitu kode status dan body yang benar-benar dikirim server (seperti pada Gambar 41 sampai Gambar 46).

[10] Bagaimana perubahan penyimpanan ke database memengaruhi kode dan pengujian?

Repository menjadi asinkron sehingga hasil perlu ditunggu sampai handler. Kontrak publik, penanganan kegagalan koneksi, dan integritas transaksi harus diuji ulang, sedangkan data tidak lagi hilang saat restart.

[11] Mengapa array tidak cocok untuk data operasional yang harus bertahan?

Array berada di memori proses sehingga hilang ketika server berhenti atau dimulai ulang, seperti terbukti ketika buku ID 4 menjadi 404 setelah restart.

[12] Mengapa tombol atau dokumentasi yang disembunyikan tidak menggantikan otorisasi server?

Pihak lain tetap dapat mengirim request langsung ke endpoint tanpa melalui tampilan. Pembatasan akses harus diperiksa oleh server pada setiap permintaan.

3. Ringkasan Hasil Uji

Nomor skenario mengikuti matriks pada modul. Hasil aktual dan status diisi dari percobaan sendiri.

Tabel 2. Matriks pengujian Praktikum 05

No	Skenario	Hasil yang diharapkan	Hasil aktual	Status	Bukti
01	Versi dan paket.	Node 24.x; dua paket sesuai modul.	Node v24.21.0 lalu v24.22.2; npm 11.19.1; express@5.3.0 dan swagger-ui-express@5.0.1.	Lulus	Gambar 1, Gambar 3, Gambar 7

No	Skenario	Hasil yang diharapkan	Hasil aktual	Status	Bukti
02	GET daftar awal.	200; array dua buku.	200; dua buku (ID 1 tersedia true, ID 2 tersedia false).	Lulus	Gambar 12, Gambar 14
03	GET detail ID 2.	200; tersedia false.	200; ID 2 *Dasar Web Service*, tersedia false.	Lulus	Gambar 16
04	Pencarian kotlin.	200; buku ID 1.	200; satu buku ID 1.	Lulus	Gambar 17
05	Pencarian tidakada.	200; array kosong.	200; data [].	Lulus	Gambar 18
06	POST valid.	201; ID baru dan Location.	201; ID 3; Location /api/buku/3.	Lulus	Gambar 19
07	GET lokasi hasil POST.	200; data baru terbaca.	200; buku ID 3 terbaca.	Lulus	Gambar 20
08	PUT lengkap.	200; ID tetap, data berubah.	200; ID 3 tetap; judul “Belajar API HTTP”, tersedia false.	Lulus	Gambar 21, Gambar 22
09	DELETE data baru.	204; body kosong.	204; body kosong.	Lulus	Gambar 23
10	GET sesudah DELETE.	404 BOOK_NOT_FOUND.	404 BOOK_NOT_FOUND.	Lulus	Gambar 24
11	ID 1abc atau 01.	400 INVALID_ID.	abc, 1abc, dan 01 semuanya 400 INVALID_ID.	Lulus	Gambar 25, Gambar 26, Gambar 27
12	ID 999.	404 BOOK_NOT_FOUND.	404 BOOK_NOT_FOUND.	Lulus	Gambar 28
13	tersedia berupa string.	400 INVALID_BOOK.	400 INVALID_BOOK.	Lulus	Gambar 29
14	PUT parsial.	400; data target tidak berubah.	400 INVALID_BOOK; ID 1 tetap “Dasar Pemrograman Kotlin”.	Lulus	Gambar 30, Gambar 34
15	JSON sengaja rusak.	400 INVALID_JSON.	400 INVALID_JSON.	Lulus	Gambar 31
16	Media type text/plain.	415 UNSUPPORTED_MEDIA_TYPE.	415 UNSUPPORTED_MEDIA_TYPE.	Lulus	Gambar 32
17	q dikirim dua kali.	400 INVALID_QUERY.	400 INVALID_QUERY.	Lulus	Gambar 35
18	Path tidak dikenal.	404 ROUTE_NOT_FOUND.	404 ROUTE_NOT_FOUND.	Lulus	Gambar 33
19	GET /openapi.json.	200; OpenAPI dan skema tersedia.	Swagger UI memuat definisi dari /openapi.json (tercatat pada log); skema Buku, BukuInput, BukuResponse, DaftarResponse, ErrorResponse tampil.	Lulus	Gambar 48, Gambar 47
20	Halaman /docs/.	Lima operasi tampil dan dapat dicoba.	Lima operasi tampil dan dapat dicoba dengan Try it out; deskripsi API dilengkapi pada Tugas 2.	Lulus	Gambar 40

No	Skenario	Hasil yang diharapkan	Hasil aktual	Status	Bukti
21	CRUD dari Swagger UI.	Status sesuai; pakai ID yang sama.	GET ID 1 = 200; ID 999 = 404; POST = 201 (ID 3); GET ID 3 = 200; PUT = 200; DELETE = 204. GET sesudah DELETE dibuktikan pada skenario 10.	Lulus	Gambar 41– Gambar 46
22	Server dihentikan.	Client tidak menerima respons HTTP.	Client menampilkan “Tidak memperoleh respons HTTP lengkap: TypeError”; tanpa status HTTP.	Lulus	Gambar 51
23	Server dimulai ulang.	Data awal kembali; data tambahan hilang.	GET /api/buku berisi dua buku awal; GET /api/buku/4 = 404.	Lulus	Gambar 52, Gambar 53
24	Instalasi sesuai README.	npm ci, generator, dan server dapat dijalankan ulang.	npm ci berhasil (68 paket, 0 kerentanan); npm run docs membuat openapi.json; npm start berjalan; GET ID 1 = 200.	Lulus	Gambar 56– Gambar 59

4. Kendala dan Penyelesaian

Tabel 3. Kendala yang dialami

Kendala	Penyebab dan penyelesaian	Bukti
npm start gagal dengan pesan EADDRINUSE setelah dokumentasi diperbarui.	Penyebab: server sebelumnya masih berjalan pada port 3200 sehingga port sudah dipakai. Penyelesaian: server lama dihentikan lalu dijalankan kembali sehingga dokumentasi terbaru tampil.	Gambar 69, Gambar 70

5. Kesimpulan

1. API katalog buku dengan Express berhasil dirancang dan dijalankan pada http://127.0.0.1:3200 dengan lima operasi CRUD yang sesuai kontrak.
2. Pemisahan router, controller, service, dan repository bekerja sesuai peran; input salah ditolak dengan status dan kode kesalahan yang tepat (400, 404, 415) tanpa mengubah data.
3. Dokumentasi OpenAPI dan Swagger UI lokal berhasil dipasang dan dipakai untuk menjalankan lima operasi dengan request nyata, lalu dilengkapi dengan deskripsi API dan contoh respons kesalahan (400, 413, 415, 500) pada Tugas 2.
4. Penyimpanan array bersifat sementara: data tambahan hilang setelah restart, dan server mati menghasilkan kegagalan koneksi tanpa status HTTP.

5. Proyek dapat dijalankan ulang melalui `npm ci`, `npm run docs`, dan `npm start`.